



河南大學
HENAN UNIVERSITY

计算机组成原理

主讲人：胡 萍

明德，新民

止于至善



第6章 计算机的运算方法



本章内容

- ★ 6.1 无符号数和有符号数
- ★ 6.2 数的定点表示和浮点表示
- ★ 6.3 定点运算
- ★ 6.4 浮点四则运算
- 6.5 算术逻辑单元





6.1 无符号数和有符号数

□ 6.1.1 无符号数

□ 6.1.2 有符号数



6.1.1 无符号数

- 机器字长：寄存器的位数；
- 无符号数：没有符号的数；
- 寄存器位数反映无符号数的表示范围。



8 位 $[0, 255]$ 或 $[0, 2^8-1]$



16 位 $[0, 65535]$ 或 $[0, 2^{16}-1]$

$n+1$ 位寄存器存储无符号整数的范围是：

$n+1$ 位寄存器的模是 2^{n+1}

$[0, 2^{n+1}-1]$



无符号数溢出举例

产生溢出时，可根据溢出方向，
采用模数纠偏法。

```
main()  
{  
    unsigned char uc1 = 255, uc2, uc3 = 3, uc4;  
    uc2 = uc1 + 2;  
    uc4 = uc3 - 5;  
    printf("uc1: 机器码 = %X H, 真值 = %u\n", uc1, uc1);  
    printf("uc2: 机器码 = %X H, 真值 = %u\n", uc2, uc2);  
    printf("uc3: 机器码 = %X H, 真值 = %u\n", uc3, uc3);  
    printf("uc4: 机器码 = %X H, 真值 = %u\n", uc4, uc4);  
}
```

```
uc1: 机器码 = FF H, 真值 = 255  
uc2: 机器码 = 1 H, 真值 = 1  
uc3: 机器码 = 3 H, 真值 = 3  
uc4: 机器码 = FE H, 真值 = 254
```

1. 计算机中，8位无符号数的表示范围是()。

B

A

0-127

B

0-255

C

0-511

D

0-63

提交





6.1 无符号数和有符号数

□ 6.1.1 无符号数

□ 6.1.2 有符号数



6.1.2 有符号数

1. 机器数与真值

4位真值 x

带符号的数

+ 1100

- 1100

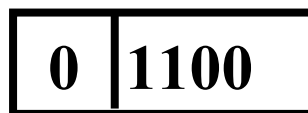
+ 0.1011

小数点后有4位

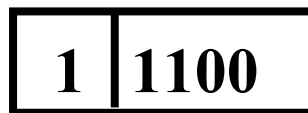
- 0.1011

机器数（机器内部使用）

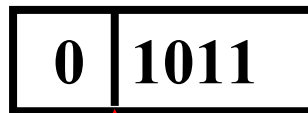
符号数字化的数



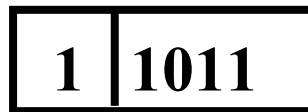
← 小数点的位置



← 小数点的位置



← 小数点的位置



← 小数点的位置



2. 原码表示法 sign and magnitude 带符号的绝对值表示

n 位整数 x 原码

$$[x]_{\text{原}} = \begin{cases} 0, |x| & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

$[x]_{\text{原}}$ 是 $n+1$ 位

如 $x = +111$ $[x]_{\text{原}} = 0, 111$ 可用“,”将符号位和数值部分隔开

$y = -111$ $[y]_{\text{原}} = 2^3 - (-111) = 1, 111$

思考：整数0的4位原码？ 整数 $0 = +000 = -000$ $[+000]_{\text{原}} = 0, 000$

缺点：整数0的原码占两组代码。 $[-000]_{\text{原}} = 1, 000$

$n+1$ 位寄存器存储整数的原码，其对应真值的表示范围是：

$$[-(2^n - 1), +(2^n - 1)]$$



2. 原码表示法

小数 x 原码

$$[x]_{\text{原}} = \begin{cases} |x| & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases} \quad x \text{ 小数点后有 } n \text{ 位}$$

如 $x = +0.111$ $[x]_{\text{原}} = 0.111$ 用“.”将符号位和数值部分隔开

$y = -0.111$ $[y]_{\text{原}} = 1 - (-0.111) = 1.111$

思考：小数0.000的原码？ $0.000 = +0.000 = -0.000$ $[+0.000]_{\text{原}} = 0.000$

缺点：小数0的原码占两组代码。 $[-0.000]_{\text{原}} = 1.000$

$n+1$ 位寄存器存储小数的原码，其对应真值的表示范围是：

$$[-(1-2^{-n}), +(1-2^{-n})]$$



2. 原码表示法

能否只作**加法**？

原码的特点：简单、直观

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

找到一个**与负数等价的正数**来代替这个负数
就可使 减→加



3. 补码表示法

(1) 补的概念

• 时钟

逆时针

$$\begin{array}{r} 6 \\ - 3 \\ \hline 3 \end{array}$$

顺时针

$$\begin{array}{r} 6 \\ + 9 \\ \hline 15 \\ - 12 \\ \hline 3 \end{array}$$

可见 -3 可用 +9 代替 减法 → 加法

称 +9 是 -3 以 12 为模的 **补数**

记作 $-3 \equiv +9 \pmod{12}$

同理 $-4 \equiv +8 \pmod{12}$

时钟以 12 为模

2. 用补码取代原码的目的是为了寻找负数的等价正数，从而将减法运算转换为加法运算。()。

A

A

对

B

错

提交





(2) 结论

□ 一个负数可用其正补数来代替，其正补数为负数加上“模”

如：4位计数器（模16）如何使：1011→0000？

$$\begin{array}{r} 1011 \\ - 1011 \\ \hline 0000 \end{array} \quad \begin{array}{r} 1011 \\ + 0101 \\ \hline 10000 \end{array}$$

自然去掉1 0000

可见 -1011 可用 + 0101 代替，记作 $-1011 \equiv + 0101 \pmod{2^4}$

同理 $-011 \equiv + 101 \pmod{2^3}$

$-0.1001 \equiv + 1.0111 \pmod{2}$

□ 一个正数和一个负数互为补数，它们绝对值之和为模。

□ 正数的补数为其本身



3. 4位二进制数 **-1010** 的补数是()。

B

A +0101

B +0110

C -0101

D -0110

提交



目前计算机中普遍采用补码表示有符号整数，如C语言中的char、short、int、long都采用补码表示。

(3) 补码定义 Two's complement

n 位整数 x 补码

$$[x]_{\text{补}} = \begin{cases} 0, & |x| \leq 2^n - 1 \\ 2^{n+1} + x, & -2^n \leq x < 0 \end{cases} \pmod{2^{n+1}}$$

$[x]_{\text{补}}$ 是 $n+1$ 位

特点：表示形式唯一。正数符号位为0，负数符号位为1。

如 $x = +111$ $[x]_{\text{补}} = 0,111$ 用“,”将符号位和数值部分隔开

$y = -111$ $[y]_{\text{补}} = 2^{3+1} + (-111) = 10000 - 111 = 1,001$

思考：-8的4位补码？ $[-8]_{\text{补}} = 2^{3+1} + (-1000) = 10000 - 1000 = 1,000$

$n+1$ 位寄存器存储整数的补码，其对应真值的表示范围是：

$$[-2^n, 2^n - 1]$$



产生溢出时，可根据溢出方向，
采用模数纠偏法。

补码溢出举例

```
main()
```

```
{
```

机器码赋值

```
char a=127,b=128,c=257;
```

```
printf("\n%d ",a);
```

```
printf("\n%d ",b);
```

```
printf("\n%d ",c);
```

```
return(0);
```

真值输出

```
}
```

127

-128

1

变量a, b, c 机器码实际存储值是多少?

4. -8的4位补码是 **1000** 。

-8的5位补码 **11000** , +8的5位补码是 **01000** ,

-8的6位补码 **111000** , +8的6位补码是 **001000**

补码表示中的符号位扩展：位数少的补码符号位向左扩展。

作答





(3) 定义

小数 x 补码

$$[x]_{\text{补}} = \begin{cases} |x| & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \end{cases} \quad (x \text{ 小数点后有 } n \text{ 位}) \quad (\text{mod } 2)$$

如 $x = +0.111$ $[x]_{\text{补}} = 0.111$ 用“.”将符号位和数值部分隔开

$$y = -0.111 \quad [y]_{\text{补}} = 2 + (-0.111) = 10.000 - 0.111 = 1.001$$

思考：-1.000的4位补码？ $[-1.000]_{\text{补}} = 2 + (-1.000) = 10.000 - 1.000 = 1.000$

$n+1$ 位寄存器存储小数的补码，其对应真值的表示范围是：

$$[-1, 1-2^{-n}]$$



思考：负数的原码和补码的关系，设 3 位真值 $x = -x_3x_2x_1$ ($-7 \leq x \leq -1$)。

解：

$$[x]_{\text{原}} = 1x_3x_2x_1$$

$$[x]_{\text{补}} = 2^4 + x = 1\ 0000 - x_3x_2x_1 = \boxed{1\ \bar{x}_3\ \bar{x}_2\ \bar{x}_1} + 1$$

$$\begin{array}{r} 1\ \overset{1}{0}\ \overset{1}{0}\ \overset{1}{0}\ \overset{2}{0} \\ - \\ \quad x_3\ x_2\ x_1 \end{array}$$

中间产物：反码

$$1\ \bar{x}_3\ \bar{x}_2\ (\bar{x}_1+1)$$

负数补码的结构？

负数补码：从低位向高位找到第一个“1”，此“1”左边为反码区，此“1”右边（包括1在内）为原码区。

$$[x]_{\text{补}} = 1, \text{ 反码 } \underline{1\ 0\dots 0}$$

原码



4. 反码表示法 One's complement

n 位整数 x 反码

$$[x]_{\text{反}} = \begin{cases} 0, |x| & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \end{cases} \pmod{2^{n+1} - 1}$$

$[x]_{\text{反}}$ 是 $n+1$ 位

如 $x = +111$ $[x]_{\text{反}} = 0,111$ 用“,”将符号位和数值部分隔开

$y = -111$ $[y]_{\text{反}} = (2^{3+1} - 1) - 111 = 1111 - 111 = 1,000$

思考：整数0的4位反码？ 整数 $0 = +000 = -000$ $[+000]_{\text{反}} = 0,000$

缺点：整数0的反码占两组代码。 $[-000]_{\text{反}} = 1,111$

$n+1$ 位寄存器存储整数的反码，其对应真值的表示范围是：

$$[-(2^n - 1), +(2^n - 1)]$$



4. 反码表示法

小数 x 反码

$$[x]_{\text{反}} = \begin{cases} |x| & 1 > x \geq 0 \\ (2 - 2^{-n}) + x & 0 \geq x > -1 \pmod{2-2^{-n}} \end{cases}$$

x 小数点后有 n 位

如 $x = +0.111$ $[x]_{\text{反}} = 0.111$

$$y = -0.111 \quad [y]_{\text{反}} = (2 - 2^{-3}) - 0.111 = 1.111 - 0.111 = 1.000$$

思考：小数 0.000 的反码？ $0.000 = +0.000 = -0.000$ $[+0.000]_{\text{反}} = 0.000$

缺点：小数 0 的反码占两组代码。 $[-0.000]_{\text{反}} = 1.111$

$n+1$ 位寄存器存储小数的反码，其对应真值的表示范围是：

$$[-(1-2^{-n}), +(1-2^{-n})]$$

整数 -2^n 的 $n+1$ 位补码、小数 $-1.0\cdots 0$ 的补码只能用定义求。

由真值求补码

- 当真值为 **正** 时，补码 等于其 **原码**。
- 当真值为 **负** 时，若其原码存在，其补码可用 **原码除符号位外每位取反，末位加 1** 求得。

例，设 $x = -1010$ ，求其补码。 解： $[x]_{\text{原}} = 1, 1010$

$$\begin{array}{r} 1, 1010 \\ + \quad 0101 \\ + \quad 1 \end{array}$$

$$[x]_{\text{补}} = 1, 0110$$

也可用负数补码结构快速求解

- 当真值为 **负** 时，若其原码不存在，必须用定义求。

例，求 $x = -8$ 的 **4** 位补码。

$$\text{解： } x = -1000 \quad [x]_{\text{补}} = 2^4 + (-1000) = 1000$$



由补码求真值

已知 n 位补码: $[x]_{\text{补}} = x_n x_{n-1} \dots x_1$

□ 当 $x_n = 0$ 时，为 **正** 数，其二进制真值 $x = x_{n-1} \dots x_1$;

□ 当 $x_n = 1$ 时，为 **负** 数，可用以下方法求真值 x ：

■ 方法1：用**定义**求真值。若其原码不存在，只能用定义

例， $[x]_{\text{补}} = 1000$ ，求 x 。

解：由定义得 $x = [x]_{\text{补}} - 2^4 = 1000 - 1\ 0000 = -1000$

■ 方法2：若其原码存在，将**补码除符号位外每位取反，末位加 1** 求得其原码，然后再求真值。

■ 方法3：若其原码存在，利用负数补码结构 $[x]_{\text{补}} = 1$ ，反码 **1 0...0**，
求得其原码，然后再求真值。
原码

方法2: 当真值为负且其原码存在, 将补码除符号位外每位取反, 末位加1可得其原码

例 已知 $[x]_{\text{补}} = 1,0001$ 求 x

解 方法1: 由定义得 $x = [x]_{\text{补}} - 2^5 = 1,0001 - 10\,0000 = -1111$

方法2: $[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}} \quad [x]_{\text{原}} = 1,1111$

$$\therefore x = -1111$$

方法3: 由负数补码结构, 可得 $[x]_{\text{原}} = 1,1111$

$$\therefore x = -1111$$

方法3: 若其原码存在, 利用负数补码结构 $[x]_{\text{补}} = 1$, 反码 1 0...0
求得其原码, 然后再求真值。原码

方法2: 当真值为负且其原码存在, 将补码除符号位外每位取反, 末位加1可得其原码

例 已知 $[x]_{\text{补}} = 1.0001$ 求 x

解 方法1: 由定义得 $x = [x]_{\text{补}} - 2 = 1.0001 - 10 = -0.1111$

方法2: $[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}} \quad [x]_{\text{原}} = 1.1111$

$\therefore x = -0.1111$

方法3: 由负数补码结构, 可得 $[x]_{\text{原}} = 1.1111$

$\therefore x = -0.1111$

方法3: 若其原码存在, 利用负数补码结构 $[x]_{\text{补}} = 1.$ 反码 1 0...0
求得其原码, 然后再求真值。原码

5. 计算机内, 二进制数-1010 的 8 位补码是()。

A 0000 0101

B

B 1111 0110

C 1000 1010

D 0000 1010

提交



6. 小数 -1.000 的 4 位补码是()。

A 1.111

B 1.000

C 1.101

D 不存在

B

提交





例6.1 设机器数字长为 8 位（其中 1 位为符号位），对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

“0”在补码中只有一种表示形式，故补码比原码和反码多表示一个负数。

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	+0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

7. 一个8位二进制整数，若采用补码表示，且由4个1和4个0组成，则最小值为()。

A -120

B -7

C -112

D -121

D

提交



8. 某机字长8位，采用补码形式(其中1位为符号位)，则机器数所能表示的范围是()。

C

A

[-127,+127]

B

[-128,+128]

C

[-128,+127]

D

[-127,+128]

提交

9. 将一个十进制数 $x = -8192$ 表示成补码时, 至少采用()位二进制代码。

B

A

13

B

14

C

15

D

12

提交





10. 设 x 为整数, $[x]_{\text{补}}=1, x_1x_2x_3x_4x_5$, 若要 $x < -16$, 应满足的条件是()。

C

A

 $x_1 \sim x_5$ 至少有一个为1

B

 x_1 必须为0, $x_2 \sim x_5$ 至少有一个为1

C

 x_1 必须为0, $x_2 \sim x_5$ 任意

D

 $x_1 \sim x_5$ 任意

提交



11. 补码 1011 0110 对应的真值是()。

A

A

-74

B

-54

C

-68

D

-48

提交



12. 【2016-13 数据类型转换】考虑以下C语言代码：

```
short si = -32767;
```

```
unsigned short usi = si;
```

D

执行上述程序段后，usi的值是（）

A -32767

B 32767

C 32768

D 32769

提交





13. 【2025-12 数据类型转换】在 32 位计算机上执行下列 C 语言代码段后, ui 的值是 ()

```
short si = -32767;
```

D

```
unsigned int usi = si;
```

A $2^{15} - 1$

B $2^{15} + 1$

C $2^{32} - 2^{15} - 1$

D $2^{32} - 2^{15} + 1$

提交



5. 移码表示法

Biased notation

补码表示很难直接判断其真值大小

十进制

二进制

补码

$$x = +31$$

$$+11111$$

$$0,11111$$

$$x = -31$$

$$-11111$$

$$1,00001$$



大

错

$$x + 2^5$$

$$+11111 + 100000$$

$$= 111111$$

$$-11111 + 100000$$

$$= 000001$$



大

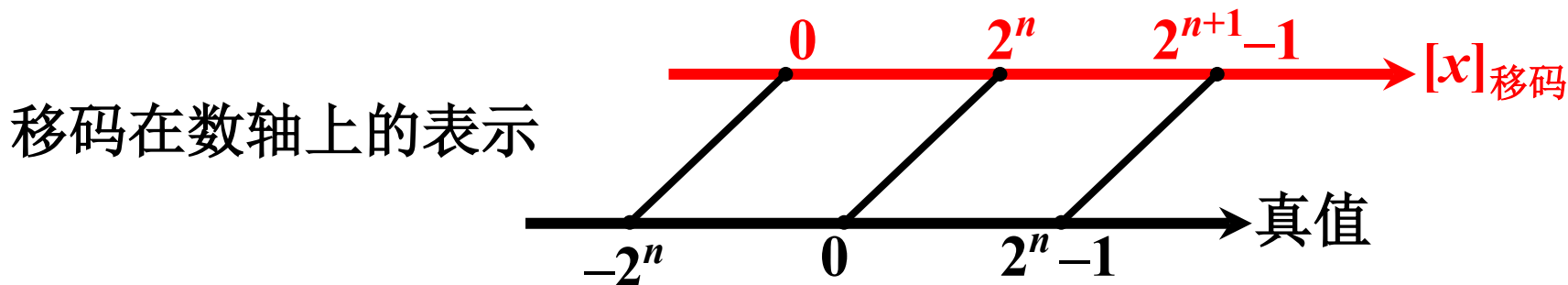
正确



(1) 移码定义

$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

x 为真值, n 为整数的位数



如 $x = 10100$

$$[x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

$x = -10100$

$$[x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

用逗号将符号位和
数值部分隔开



(2) 移码和补码的比较

设 $x = +1100100$

$$[x]_{\text{移}} = 2^7 + 1100100 = \mathbf{1},1100100$$

$$[x]_{\text{补}} = \mathbf{0},1100100$$

补码与移码只差一个符号位

设 $x = -1100100$

$$[x]_{\text{移}} = 2^7 - 1100100 = \mathbf{0},0011100$$

$$[x]_{\text{补}} = \mathbf{1},0011100$$



(4) 移码的特点

□ 当 $x = 0$ 时 $[+0]_{\text{移}} = 2^5 + 0 = 1,00000$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

$$\therefore [+0]_{\text{移}} = [-0]_{\text{移}}$$

□ 当 $n = 5$ 时 最小的真值为 $-2^5 = -100000$

$$[-100000]_{\text{移}} = 2^5 - 100000 = 000000$$

可见，最小真值的移码为全 0

用移码表示浮点数的阶码，能方便地判断浮点数的阶码大小



14. () 表示法主要用于表示浮点数中的阶码。

A 原码

B 补码

C 移码

D 反码

C

提交





机器码小结

■ 4类机器码应用

- 原码 用来表示浮点（实）数的尾数
- 反码 已不用于表示数值数据
- 补码 50年代开始成为整数标准
- 移码 用于浮点数阶码

■ 补码优势

- 模运算，加、减运算统一
- 唯一0，方便使用



6.2 数的定点表示和浮点表示

□ 6.2.1 定点表示

□ 6.2.2 浮点表示

□ 6.2.3 定点数和浮点数的比较

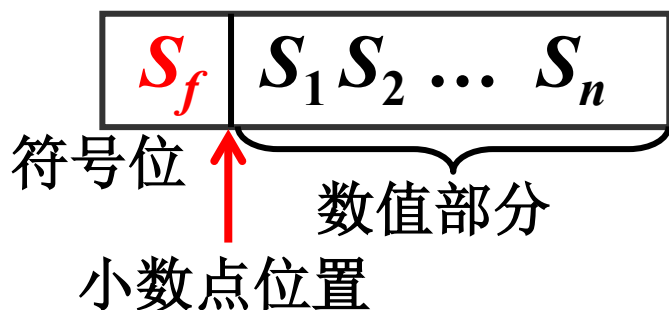
□ 6.2.4 举例

□ 6.2.5 IEEE 754 标准



6.2.1 定点表示 小数点按约定方式标出

定点小数



定点整数

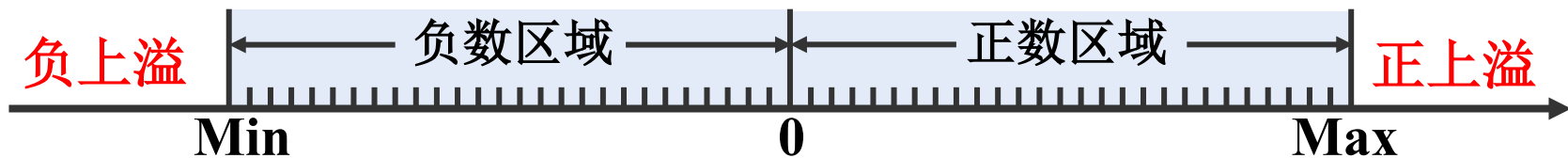


	定点小数范围	定点整数范围
原码	$[-(1 - 2^{-n}), (1 - 2^{-n})]$	$[-(2^n - 1), (2^n - 1)]$
反码	$[-(1 - 2^{-n}), (1 - 2^{-n})]$	$[-(2^n - 1), (2^n - 1)]$
补码	$[-1, (1 - 2^{-n})]$	$[-2^n, (2^n - 1)]$
移码	-----	$[-2^n, (2^n - 1)]$



● 定点数的表示范围

- 表示范围中的每个数都能对应数轴上的一个刻度，刻度在数轴上均匀分布。定点整数的最小刻度间距是1，定点小数为 2^{-n} 。
- 数据超出表示范围时产生溢出：当数据大于最大正数时，发生正上溢；当数据小于最小负数时，发生负上溢。
- 定点小数还存在精度问题，所有不在数轴刻度上的纯小数都超出了能表示的精度，无法表示，此时发生精度溢出，只能采用舍入的方式近似表示。



15. 在小数定点机中，下列描述正确的是()。 A

A

原码和反码不能表示 -1 ，补码可以表示 -1

B

原码、反码和补码均可以表示 -1

C

原码、反码和补码可以表示 -1 ，且三种机器数的表示范围相同

D

原码和反码可以表示 -1 ，补码不能表示 -1

提交

16. 9位原码能表示的数据的个数是()。

C

A

10

B

9

C

511

D

512

提交





6.2 数的定点表示和浮点表示

□ 6.2.1 定点表示

□ 6.2.2 浮点表示

□ 6.2.3 定点数和浮点数的比较

□ 6.2.4 举例

□ 6.2.5 IEEE 754 标准



6.2.2 浮点表示 为什么引入浮点数表示？

浮点数的一般形式： $N = S \times r^j$ S 尾数 j 阶码 r 基数

计算机中 r 取 2、4、8、16 等

例：当 $r = 2$ $N = 11.0101$

二进制表示

规格化数 ✓ = 0.110101×2^{10} 规格化数能保证
数的最大精度

$$= 1.10101 \times 2^1$$

$$= 1101.01 \times 2^{-10}$$

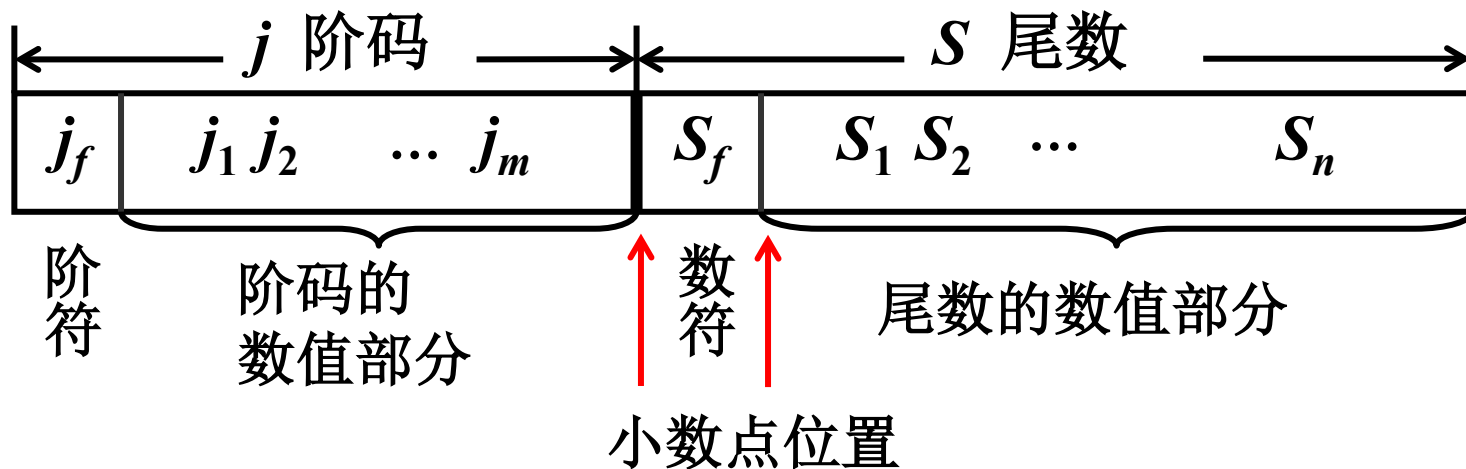
$$✓ = 0.00110101 \times 2^{100}$$

S 纯小数、可正可负 j 整数、可正可负



1. 浮点数的表示形式

浮点数实际上是用一对定点数（阶码和尾数）来表示



S_f 代表浮点数的符号 n 其位数反映浮点数的精度

m 其位数反映浮点数的表示范围

j_f 和 m 共同表示小数点的实际位置

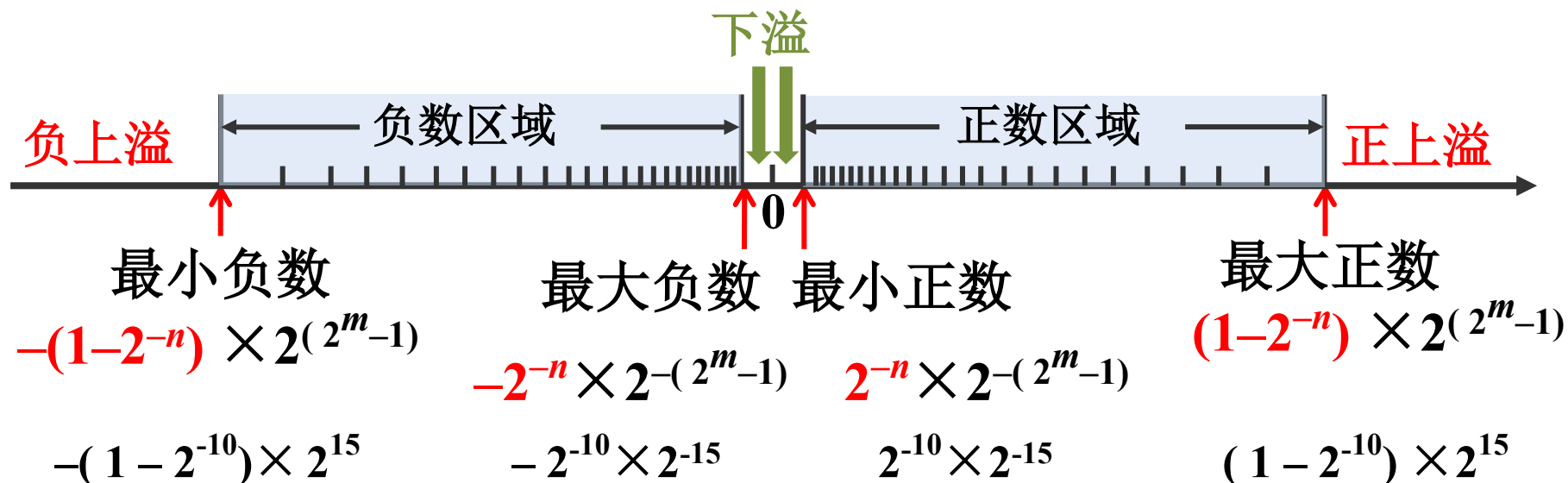
设浮点数**未规格化**，基数 $r=2$ ，阶码数值部分为 m 位，尾数数值部分为 n 位，阶码和尾数符号各占 1 位。

2. 浮点数的表示范围

$$\text{浮点数 } N = S \times 2^j$$

□ 上溢 阶码 > 最大阶码

□ 下溢 阶码 < 最小阶码 按机器零处理



若 $m=4$ $n=10$



3. 浮点数真值的规格化形式

$r = 2$ 尾数最高位为 1

$r = 4$ 尾数最高 2 位不全为 0

$r = 8$ 尾数最高 3 位不全为 0

基数不同，浮点数的
规格化形式不同



4. 浮点数的规格化

$r = 2$ 左规 尾数左移 1 位，阶码减 1

右规 尾数右移 1 位，阶码加 1

$r = 4$ 左规 尾数左移 2 位，阶码减 1

右规 尾数右移 2 位，阶码加 1

$r = 8$ 左规 尾数左移 3 位，阶码减 1

右规 尾数右移 3 位，阶码加 1

基数 r 越大，可表示的浮点数的范围越大

基数 r 越大，浮点数的精度降低



6.2 数的定点表示和浮点表示

□ 6.2.1 定点表示

□ 6.2.2 浮点表示

□ 6.2.3 定点数和浮点数的比较

□ 6.2.4 举例

□ 6.2.5 IEEE 754 标准



● 定点数和浮点数的比较

- 当位数相同时，浮点数的表示范围大得多
- 当浮点数为规格化数时，相对精度高
- 浮点运算步骤多，速度低，线路复杂
- 溢出的判断方法不同，浮点数对阶码进行判断，定点数对数值本身进行判断



● 机器零

- 当浮点数 **尾数为 0** 时，不论其阶码为何值，按**机器零**处理
- 当浮点数 **阶码等于或小于它所表示的最小 数** 时，不论尾数为何值，按**机器零**处理



6.2 数的定点表示和浮点表示

□ 6.2.1 定点表示

□ 6.2.2 浮点表示

□ 6.2.3 定点数和浮点数的比较

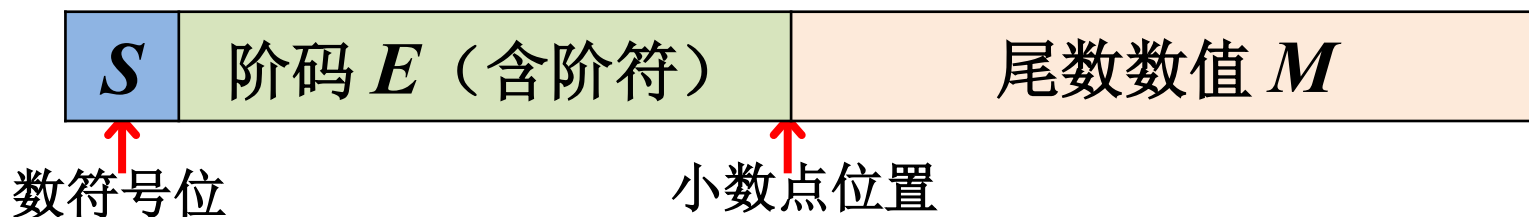
□ 6.2.4 举例

□ 6.2.5 IEEE 754 标准

■ IEEE754 标准中，规格化的32 位浮点数 x 的真值表示为：

$$x = (-1)^S \times (1.M) \times 2^{E-127} \quad e = E - 127$$

6.2.5 IEEE 754 标准



	数符 S	阶码 E	尾数 M	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80

■ 阶码 E ：用**移码**表示，短实数、长实数、临时实数的偏移量分别为7FH、3FFH和3FFFH(127、1023和16383)。

■ 尾数数值 M ：用**原码**表示，IEEE中规格化尾数为**1.M**=1.f...f，计算机实际存储时，这个1省略。



Float类型与真值之间的变换流程

浮点数真值：二/十进制数N

$$N = (-1)^S \times 2^e \times 1.M$$

$$E = e + 127$$

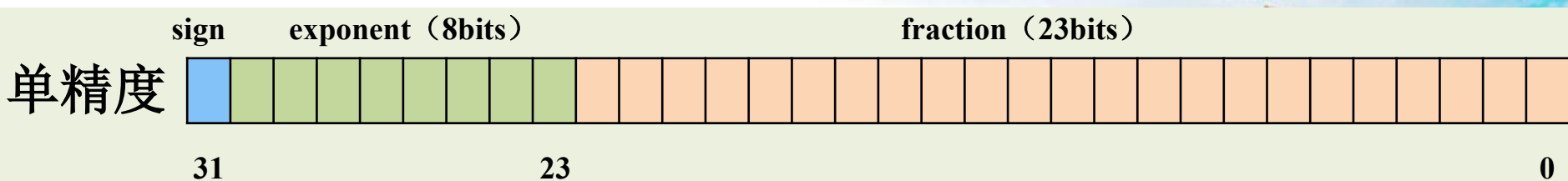
保存 S 、 E 、 M

32位单精度数 (float 类型)

$$\text{计算 } (-1)^S \times 2^e \times 1.M, \text{ 按权展开}$$

$$e = E - 127$$

从32位二进制中提取 S 、 E 、 M



【例】求单精度数 $0\ 10000110\ 011001000100000000000000$ 对应的真值。

解：32位单精度数包含：1位符号位 S ，8位阶码 E ，23位尾数 M

由符号位 $S=0$ ，可知浮点数为正数。

阶码 $E=10000110$ ，则对应的真值 $e = E - 127 = 10000110 - 1111111 = 111$

尾数 $M=011001000100000000000000$ ，其对应的真值为 $1.M =$

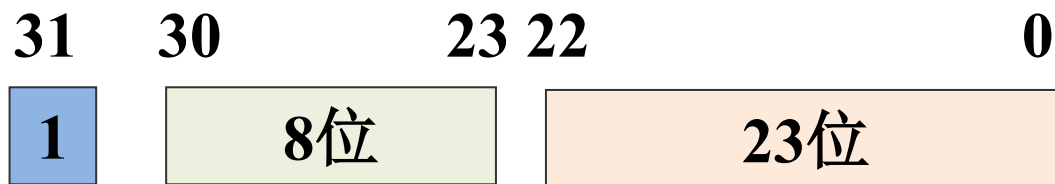
$1.011001000100000000000000$

$$\begin{aligned} \text{真值 } x &= (-1)^S \times (1.M) \times 2^{E-127} = 1.0110010001 \times 2^{111} \\ &= 10110010.001 \end{aligned}$$



浙江大学考研试题

计算机储存程序的特点之一是把数据和指令都作为二进制信号看待。今有一计算机字长**32bit**，数符位是第**31bit**；单精度浮点数格式如图所示。



对二进制数**1000 1111 1110 1111 1100 0000 0000 0000**

- ①表示一个补码整数，其十进制值是多少？
- ②表示一个无符号整数，其十进制值是多少？
- ③表示一个**IEEE754**标准的单精度浮点数，其值是多少？



● 浙江大学考研试题

二进制数 **1000 1111 1110 1111 1100 0000 0000 0000**

① 表示一个补码整数，其十进制值是多少？

解：作为补码整数，其对应的原码是：

1111 0000 0001 0000 0100 0000 0000 0000

$\therefore$ 十进制值是 $-(2^{30} + 2^{29} + 2^{28} + 2^{20} + 2^{14})$



● 浙江大学考研试题

二进制数 **1000 1111 1110 1111 1100 0000 0000 0000**

② 表示一个无符号整数，其十进制值是多少？

解：作为无符号整数，其十进制值是：

$$\begin{aligned} &2^{31} + 2^{27} + 2^{26} + 2^{25} + 2^{24} + 2^{23} + 2^{22} + 2^{21} + 2^{19} + 2^{18} + 2^{17} \\ &+ 2^{16} + 2^{15} + 2^{14} \end{aligned}$$



● 浙江大学考研试题

二进制数 **1000 1111 1110 1111 1100 0000 0000 0000**

③ 作为IEEE754标准的单精度浮点数

解：阶码是0001 1111

$$\begin{aligned}\text{指数 } e &= \text{阶码} - 127 = 0001\ 1111 - 0111\ 1111 \\ &= -1100000\end{aligned}$$

$$\text{尾数} = 110\ 1111\ 1100\ 0000\ 0000\ 0000$$

$$\begin{aligned}\text{则 } 1.M &= 1.110\ 1111\ 1100\ 0000\ 0000\ 0000 \\ &= 1.110\ 1111\ 11\end{aligned}$$



● 浙江大学考研试题

二进制数 **1000 1111 1110 1111 1100 0000 0000 0000**

③ 作为IEEE754标准的单精度浮点数

解：阶码是0001 1111

$$\begin{aligned}\text{指数 } e &= \text{阶码} - 127 = 0001\ 1111 - 0111\ 1111 \\ &= -1100000\end{aligned}$$

$$\text{尾数} = 110\ 1111\ 1100\ 0000\ 0000\ 0000$$

$$\begin{aligned}\text{则 } 1.M &= 1.110\ 1111\ 1100\ 0000\ 0000\ 0000 \\ &= 1.110\ 1111\ 11 =\end{aligned}$$

$$\therefore \text{单精度浮点数值为: } X = -(1.110\ 1111\ 11) \times 2^{-1100000}$$

19. 【2011 单精度浮点数】Float型数据通常用IEEE 754单精度浮点数格式表示。若编译器将float型变量 x 分配在一个32位浮点寄存器FR1中，且 $x=-8.25$ ，则FR1的内容是（ ）。

A

- ☒ A C104 0000H
- ☐ B C242 0000H
- ☐ C C184 0000H
- ☐ D C1C2 0000H

提交



6.3 定点运算

□ 6.3.1 移位运算

□ 6.3.2 加法与减法运算

□ 6.3.3 乘法运算

□ 6.3.4 除法运算



6.3.1 移位运算

1. 移位的意义

$15\text{ m} = 1500\text{ cm}$ 生活中，小数点右移 2 位

机器用语 15 相对于小数点 左移 2 位

(小数点不动)

数左移 绝对值扩大

数右移 绝对值缩小

在计算机中，移位与加减配合，能够实现乘除运算



6.3.1 移位运算

补充：逻辑移位

举例 设机器字长为8，无符号数 $A = 26 = 11010$

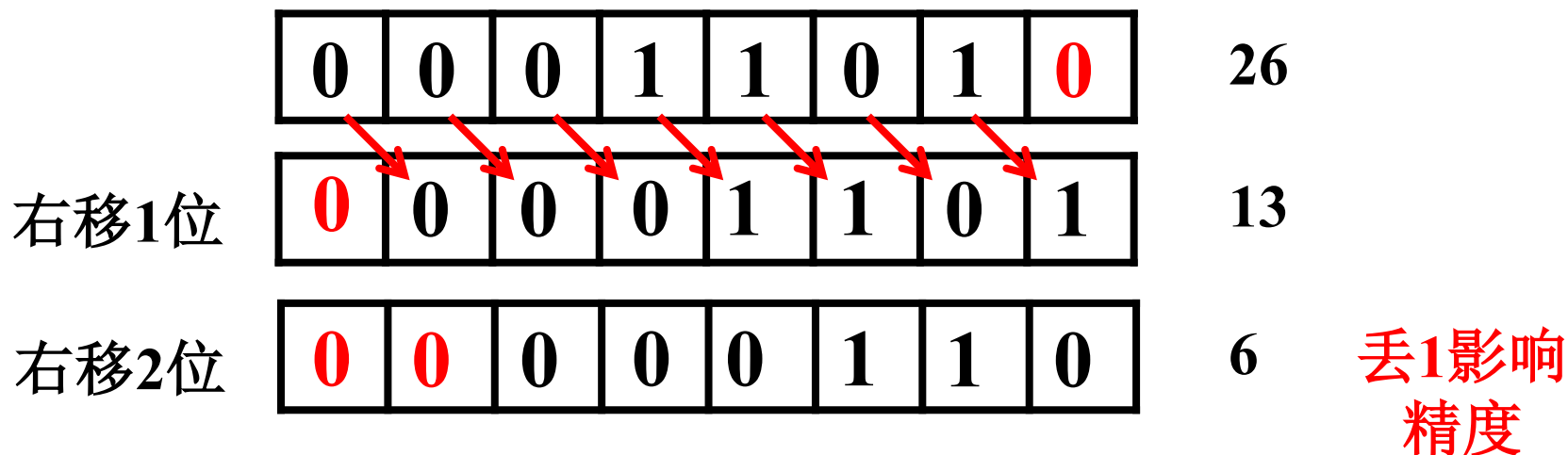
	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td></tr></table>	0	0	0	1	1	0	1	0	26	
0	0	0	1	1	0	1	0				
左移1位	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	0	0	1	1	0	1	0	0	52	
0	0	1	1	0	1	0	0				
左移2位	<table><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	1	1	0	1	0	0	0	104	
0	1	1	0	1	0	0	0				
左移3位	<table><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	1	0	1	0	0	0	0	208	
1	1	0	1	0	0	0	0				
左移4位	<table><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	1	0	0	0	0	0	160	丢1出错
1	0	1	0	0	0	0	0				



6.3.1 移位运算

补充：逻辑移位

举例 设机器字长为8，无符号数 $A = 26 = 11010$





6.3.1 移位运算

补充：逻辑移位

- 无符号数的移位运算称为逻辑移位。
- 逻辑移位产生的空位均补“0”。
- 逻辑移位左移时
 - 最高位丢0，无错误，扩大2倍
 - 最高位丢1，结果出错
- 逻辑移位右移时
 - 最低位丢0，无错误，缩小到1/2
 - 最低位丢1，影响精度



6.3.1 移位运算

2. 算术移位规则 有符号数的移位运算

负数补码：从低位向高位找到第一个“1”，此“1”左边为反码区，此“1”右边（包括1在内）为原码区。

$[x]_{\text{补}} = 1$ ，反码 $\underline{1 \ 0 \dots 0}$
 原码



6.3.1 移位运算

2. 算术移位规则 有符号数的移位运算

符号位不变

真值	码 制	添补代码	
正数	原码、补码、反码	0	{ 左移最高位丢1，结果出错 { 右移最低位丢1，影响精度
负数	原 码	0	{ 左移最高位丢1，结果出错 { 右移最低位丢1，影响精度
	补 码	左移 添 0	{ 左移最高位丢0，结果出错 { 右移最低位丢1，影响精度
		右移 添 1	
	反 码	1	{ 左移最高位丢0，结果出错 { 右移最低位丢0，影响精度



20. 不论是正数还是负数，移位后其符号位均不变。()。

A

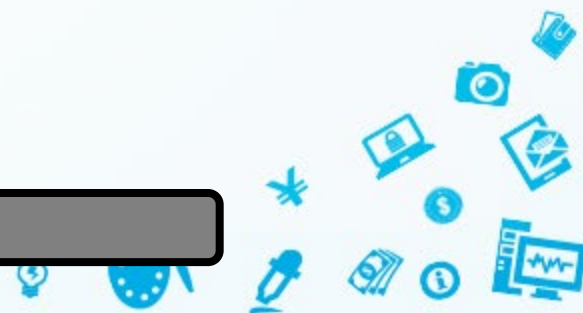
A

对

B

错

提交





● 例6.6

设机器数字长为 8 位（含 1 位符号位），写出 $A = +26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$



●例6.6

解: $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$



左移3位?

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}}$	
移位前	0, 0 0 1 1 0 1 0	+26
左移一位	0, 0 1 1 0 1 0 0	+ 52
左移两位	0, 1 1 0 1 0 0 0	+104
左移3位	0, 1 0 1 0 0 0 0	+80

正数，左移最高位丢1，结果出错



例6.6

解: $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}}$	
移位前	0, 0 0 1 1 0 1 0	+26
右移一位	0, 0 0 0 1 1 0 1	+13
右移两位	0, 0 0 0 0 1 1 0	+6

正数，右移最低位丢1，影响精度



●例6.7

设机器数字长为 8 位（含 1 位符号位），写出 $A = -26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = -26 = -11010$

则 $[A]_{\text{原}} = 1,0011010$

$$[A]_{\text{反}} = 1,1100101$$

$$[A]_{\text{补}} = 1,1100110$$



● 例6.7 解: $[A]_{\text{原}} = 1,0011010$

原码

移位操作	机 器 数	对应的真值
移位前	1 ,0011010	- 26
左移一位	1 ,011010 0	- 52
左移两位	1 ,11010 00	- 104
右移一位	1 , 0 001101	- 13
右移两位	1 , 00 00110	- 6

负数的原码，左移最高位丢1，结果出错

负数的原码，右移最低位丢1，影响精度



● 例6.7 解: $[A]_{\text{反}} = 1,1100101$

反码

移位操作	机 器 数	对应的真值
移位前	1 ,1100101	- 26
左移一位	1 ,100101 1	- 52
左移两位	1 ,00101 11	- 104
右移一位	1 , 1 110010	- 13
右移两位	1 , 1 111001	- 6

负数的反码，左移最高位丢0，结果出错

负数的反码，右移最低位丢0，影响精度



● 例6.7 解: $[A]_{\text{补}} = 1,1100110$

补码

移位操作	机 器 数	对应的真值
移位前	1 ,1100110	- 26
左移一位	1 ,100110 0	- 52
左移两位	1 ,00110 00	- 104
右移一位	1 , 1 110011	- 13
右移两位	1 , 11 11001	- 7

负数的补码，左移最高位丢0，结果出错

负数的补码，右移最低位丢1，影响精度



21. 设机器数字长8位(含1位符号位), 若机器数BAH为原码, 则算术左移一位得(), 算术右移一位得()。

C

A

F4H EDH

B

B4H 6DH

C

F4H 9DH

D

B5H EDH

提交



22. 【2018 移位运算】整数 x 的机器数为1101 1000，分别对 x 进行逻辑右移1位和算术右移1位操作，得到的机器数分别是（ ）。

- ☐ A 1110 1100、1110 1100
- ☒ B 0110 1100、1110 1100
- ☐ C 1110 1100、0110 1100
- ☐ D 0110 1100、0110 1100

提交



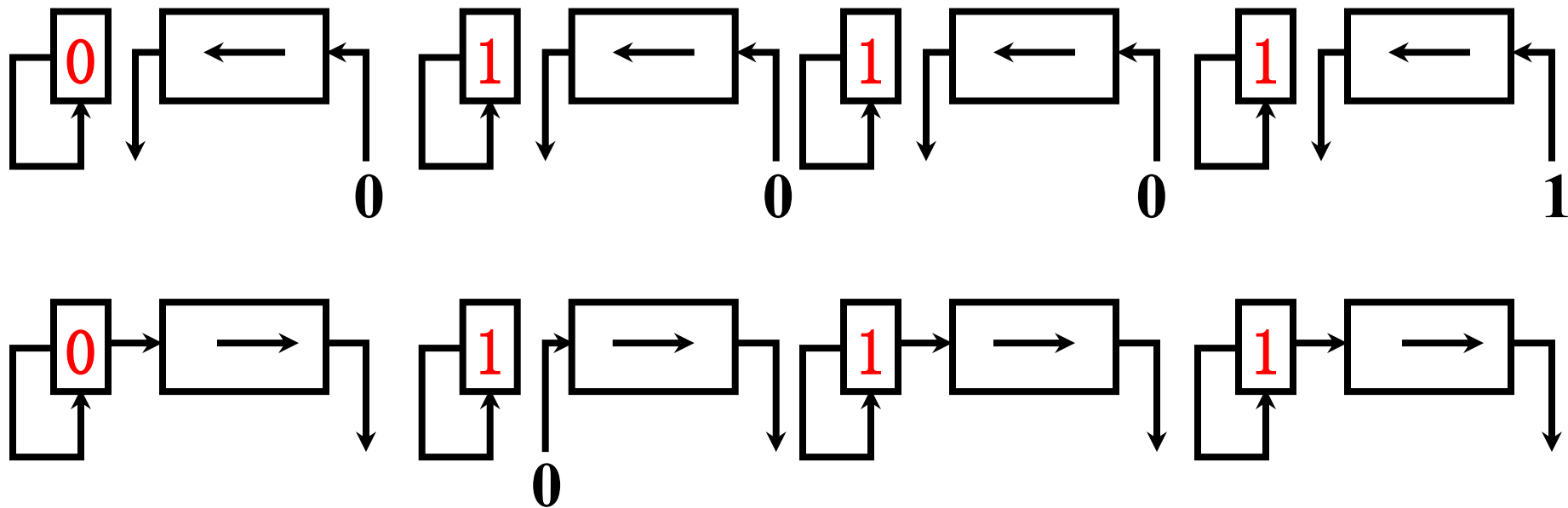
23. 【2013 移位运算】某字长为8位的计算机中，已知整型变量 x , y 的机器数分别为 $[x]_{\text{补}}=1111\ 0100$, $[y]_{\text{补}}=1011\ 0000$,若整型变量 $z=2*x+y/2$,则 z 的机器数为 () 。

- ☒ A 1100 0000
- ☐ B 0010 0100
- ☐ C 1010 1010
- ☐ D 溢出

提交



3. 算术移位的硬件实现



(a) 真值为正 (b) 负数的原码 (c) 负数的补码 (d) 负数的反码

← 丢 1 出错

出错

正确

正确

→ 丢 1 影响精度

影响精度

影响精度

正确



4. 算术移位和逻辑移位的区别

□ 算术移位：有符号数的移位

□ 逻辑移位：无符号数的移位

逻辑左移时，低位添 0，高位移丢

逻辑右移：高位添 0，低位移丢

例如

01010011

10110010

逻辑左移

10100110

逻辑右移

01011001

算术左移 (补码) 00100110

算术右移 (补码) 11011001



6.3 定点运算

□ 6.3.1 移位运算

□ 6.3.2 加法与减法运算

□ 6.3.3 乘法运算

□ 6.3.4 除法运算



6.3.2 加法与减法运算

1. 补码加减运算公式

(1) 加法

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2}$

连同符号位一起相加，符号位产生的进位自然丢掉



6.3.2 加法与减法运算

1. 补码加减运算公式

(2) 减法

$$A - B = A + (-B)$$

整数 $[A - B]_{\text{补}} = [A + (-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \quad (\text{mod } 2^{n+1})$

小数 $[A - B]_{\text{补}} = [A + (-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \quad (\text{mod } 2)$

连同符号位一起相加，符号位产生的进位自然丢掉。



已知 $[B]_{\text{补}}$ ，如何求 $[-B]_{\text{补}}$ ？

$[y]_{\text{补}}$ 连同符号位在内，每位取反，末位加 1，即得 $[-y]_{\text{补}}$

思考：该表中补码运算结果均无溢出，分析符号位的进位和最高数值位的进位状态 两者进位状态相同

● 设机器字长为4位。4位补码对应真值范围 $[-8, +7]$

真值	补码运算
$\begin{array}{r} A = +3 \\ + B = +4 \\ \hline A+B = +7 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = 0, 011 \\ \text{均无进位} + [B]_{\text{补}} = 0, 100 \\ \hline [A+B]_{\text{补}} = 0, 111 \end{array}$
$\begin{array}{r} A = -4 \\ + B = -4 \\ \hline A+B = -8 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = 1, 100 \\ \text{均有进位} + [B]_{\text{补}} = 1, 100 \\ \hline [A+B]_{\text{补}} = 11, 000 \end{array}$
$\begin{array}{r} A = +3 \\ - B = -4 \\ \hline A-B = +7 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = 0, 011 \\ \text{均无进位} + [-B]_{\text{补}} = 0, 100 \\ \hline [A-B]_{\text{补}} = 0, 111 \end{array}$
$\begin{array}{r} A = -4 \\ - B = +4 \\ \hline A-B = -8 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = 1, 100 \\ \text{均有进位} + [-B]_{\text{补}} = 1, 100 \\ \hline [A-B]_{\text{补}} = 11, 000 \end{array}$

思考：该表中补码运算结果均有溢出，分析符号位的进位和最高数值位的进位状态。 两者进位状态不同

● 设机器字长为4位。4位补码对应真值范围 $[-8, +7]$

真值	补码运算
$\begin{array}{r} A = +5 \\ + B = +4 \\ \hline A+B = +9 > +7 \end{array}$ 溢出方向	$\begin{array}{r} [A]_{\text{补}} = 0, \boxed{1}01 \\ + [B]_{\text{补}} = 0, \boxed{1}00 \\ \hline [A+B]_{\text{补}} = 1, 001 \end{array}$ 符号位无进位 最高数值位有进位 -7 +16修正
$\begin{array}{r} A = -5 \\ + B = -4 \\ \hline A+B = -9 < -8 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = \boxed{1}, 011 \\ + [B]_{\text{补}} = \boxed{1}, 100 \\ \hline [A+B]_{\text{补}} = 10, 111 \end{array}$ 符号位有进位 最高数值位无进位 +7 -16修正
$\begin{array}{r} A = +5 \\ - B = -4 \\ \hline A-B = +9 > +7 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = 0, \boxed{1}01 \\ + [-B]_{\text{补}} = 0, \boxed{1}00 \\ \hline [A-B]_{\text{补}} = 1, 001 \end{array}$ 符号位无进位 最高数值位有进位
$\begin{array}{r} A = -5 \\ - B = +4 \\ \hline A-B = -9 < -8 \end{array}$	$\begin{array}{r} [A]_{\text{补}} = \boxed{1}, 011 \\ + [-B]_{\text{补}} = \boxed{1}, 100 \\ \hline [A-B]_{\text{补}} = 10, 111 \end{array}$ 符号位有进位 最高数值位无进位



2. 溢出判断

(1) 一位符号位判溢出

方法：参加操作的 **两个数**（减法时即为被减数和“求补”以后的减数）**符号相同**，其**结果的符号与原操作数的符号不同**，即为溢出。

硬件实现溢出信号 **Overflow (OF)**:

$$\text{OF} = \text{符号位进位} \oplus \text{最高数值位进位} = 1 \quad \text{溢出}$$

如	$1 \oplus 0 = 1$	} 有 溢出	$0 \oplus 0 = 0$	} 无 溢出
	$0 \oplus 1 = 1$		$1 \oplus 1 = 0$	

24. 两补码相加，采用1位符号位，则当()时，表示结果溢出。

D

A

最高位有进位

B

最高位进位和次高位进位异或结果为0

C

最高位为1

D

最高位进位和次高位进位异或结果为1

提交





● 例 6.9 设 $A = -9$, $B = -5$
求 $[A+B]_{\text{补}}$

$$\begin{array}{rcl}
 & & \text{有进位} \quad \text{有进位} \\
 \text{解:} & [A]_{\text{补}} & = 1, 0111 \\
 & + [B]_{\text{补}} & = 1, 1011 \\
 \hline
 [A]_{\text{补}} + [B]_{\text{补}} & = 1, 1, 0010 & = [A+B]_{\text{补}} \\
 & \text{进位自然舍掉} &
 \end{array}$$

$$\therefore A+B = (-1110)_2 = -14$$

OF = 符号位进位 $\oplus$ 最高数值位进位 = $1 \oplus 1 = 0$, 无溢出。



●例 6.11 设机器数字长为 8 位（含 1 位符号位）

且 $A = -93$, $B = +45$, 用补码求 $A - B$

解: $A = -93 = -1011101 B$

$B = +45 = +0101101 B$

有进位 无进位

$[A]_{\text{补}} = 1, 0100011$

$[B]_{\text{补}} = 0, 0101101$

$+ [-B]_{\text{补}} = 1, 1010011$

$[A]_{\text{补}} + [-B]_{\text{补}} = 10, 1110110 = [A - B]_{\text{补}}$

进位自然舍掉

符号位进位 $\oplus$ 最高有效位的进位=1, 产生了溢出。



● 小数双符号位补码（以4为模，P240）

小数 x 双符号位补码

$$[x]_{\text{补}} = \begin{cases} |x| & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \pmod{4} \end{cases}$$

如 $x = +0.1110$ $[x]_{\text{补}} = 00.1110$

$y = -0.1100$ $[y]_{\text{补}} = 4 + (-0.1100) = 100.0000$

$$\begin{array}{r} - \quad 0.1100 \\ \hline 11.0100 \end{array}$$

正数符号位为00，负数符号位为11。



整数双符号位补码（以 2^{n+2} 为模）

n 位整数 x 双符号位补码

$$[x]_{\text{补}} = \begin{cases} 0, |x| & 2^n > x \geq 0 \\ 2^{n+2} + x & 0 > x \geq -2^n \end{cases} \pmod{2^{n+2}}$$

如 $x = +1110$

$[x]_{\text{补}} = 00,1110$

$y = -1100$

$[y]_{\text{补}} = 2^6 + (-1100) = 1000000$

$$\begin{array}{r} -1100 \\ \hline 11,0100 \end{array}$$

正数符号位为00，负数符号位为11。

思考：该表中补码运算结果均有溢出，双符号位如何进行溢出判断？如何修正？代价是？ **双符号位不同**

● 设机器字长为4位。4位补码对应真值范围 $[-8, +7]$

真值	补码运算	
$\begin{array}{r} A = +5 \\ + B = +4 \\ \hline A+B = +9 \end{array} > +7$	$\begin{array}{r} [A]_{\text{补}} = 00, 101 \\ + [B]_{\text{补}} = 00, 100 \\ \hline [A+B]_{\text{补}} = 01, 001 \end{array}$	最高符号位是真正符号位，其余是数值部分。
$\begin{array}{r} A = -5 \\ + B = -4 \\ \hline A+B = -9 \end{array} < -8$	$\begin{array}{r} [A]_{\text{补}} = 11, 011 \\ + [B]_{\text{补}} = 11, 100 \\ \hline [A+B]_{\text{补}} = 10, 111 \end{array}$	
$\begin{array}{r} A = +5 \\ - B = -4 \\ \hline A-B = +9 \end{array} > +7$	$\begin{array}{r} [A]_{\text{补}} = 00, 101 \\ + [-B]_{\text{补}} = 00, 100 \\ \hline [A-B]_{\text{补}} = 01, 001 \end{array}$	
$\begin{array}{r} A = -5 \\ - B = +4 \\ \hline A-B = -9 \end{array} < -8$	$\begin{array}{r} [A]_{\text{补}} = 11, 011 \\ + [-B]_{\text{补}} = 11, 100 \\ \hline [A-B]_{\text{补}} = 10, 111 \end{array}$	



2. 溢出判断

(2) 两位符号位判溢出

结果的双符号位 相同 未溢出

结果的双符号位 不同 溢出

00. × × × × ×

11. × × × × ×

10. × × × × ×

01. × × × × ×

最高符号位 是 真正的符号，其余为数值部分。



● 例 6.15 $x = +\frac{11}{16}$ $y = +\frac{7}{16}$, 用变形补码求 $x + y$

解: $x = 0.1011$, $y = 0.0111$

$$[x]_{\text{补}} = 00.1011, [y]_{\text{补}} = 00.0111$$

则 $[x + y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} = 00.1011 + 00.0111 = \boxed{01}.0010$

符号位为01, 溢出, 正确的符号为正, 其余为数值

$$x + y = +1.0010$$

25. 两补码相加，采用2位符号位，则当符号位为[**10**]和[**11**]时，表示结果溢出。当双符号补码判断有溢出时，正确的符号位为[**最高符号位**]，其余都是数值位。

最高符号位

正常使用填空题需3.0以上版本雨课堂

作答

26. 在整数定点机中，机器数采用补码，双符号位，若它的十六进制表示为C0H，它代表的真值是()。

C

- ☐ A -1
- ☐ B +3
- ☒ C -64
- ☐ D -128

提交



27. 在整数定点机中，机器数采用补码，双符号位，若它的十六进制表示为BEH，它代表的真值是()。

C

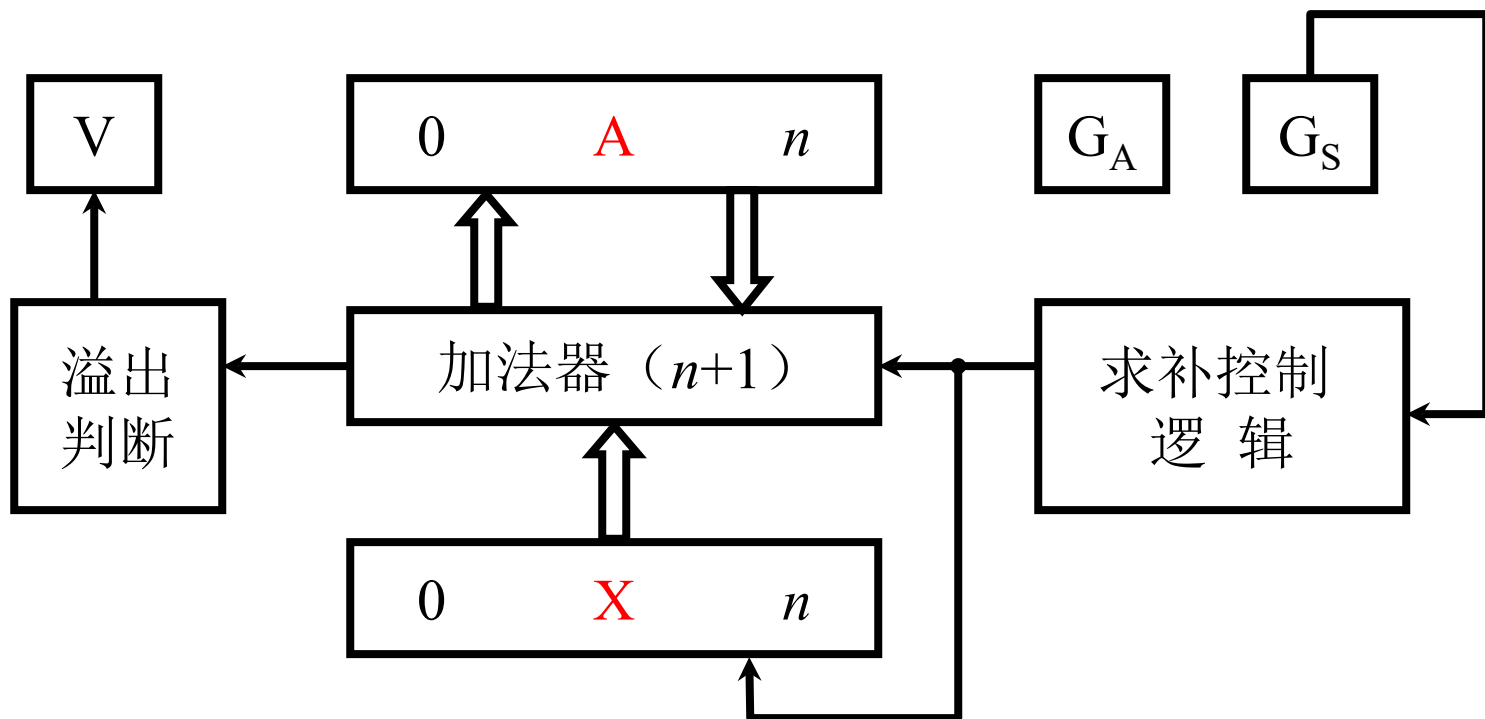
- ☐ A -1
- ☐ B +3
- ☒ C -66
- ☐ D -128

提交





3. 补码加减法的硬件配置

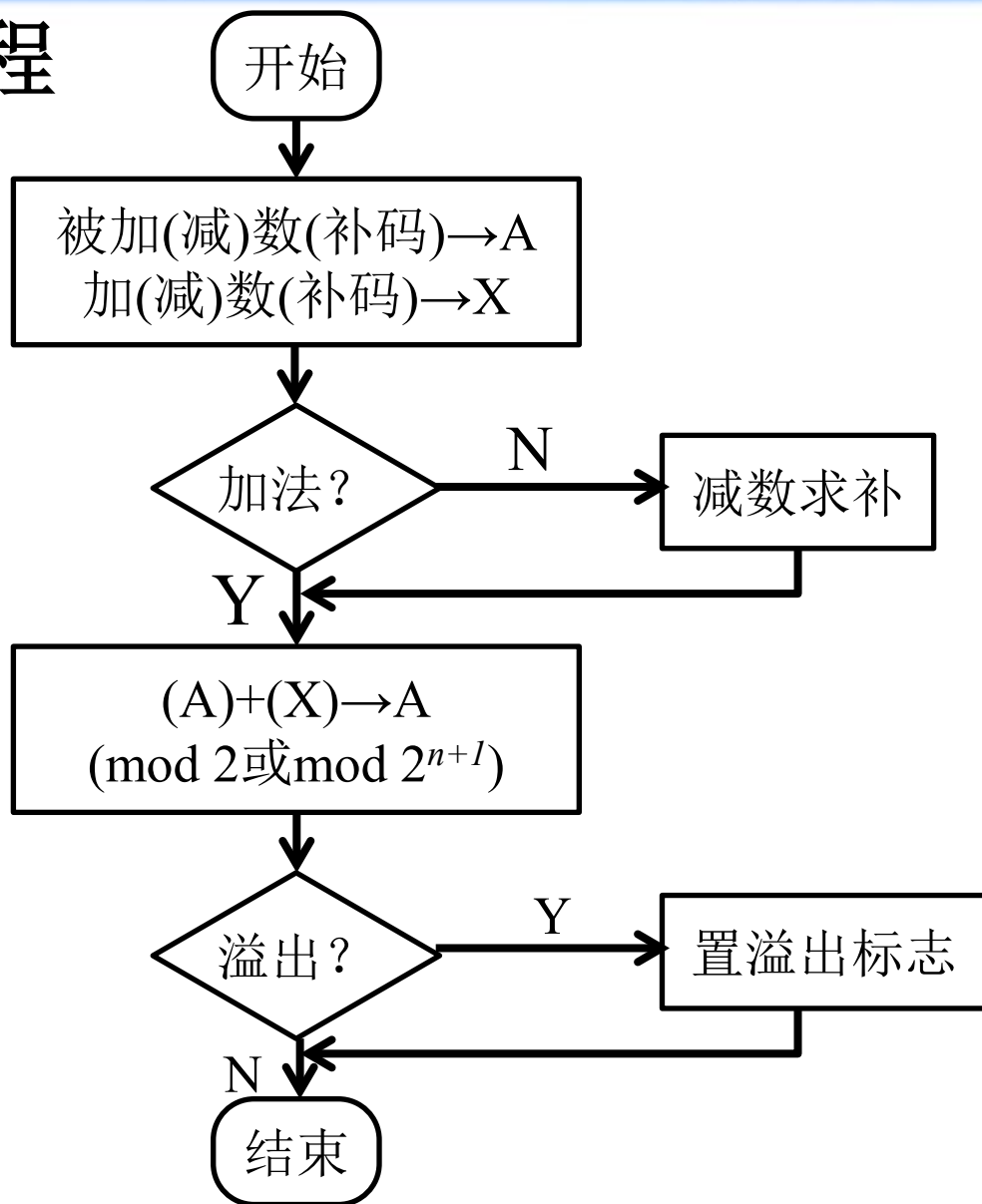


A、X 均 $n+1$ 位

用减法标记 G_S 控制求补逻辑



4.补码加减运算流程





6.4 浮点四则运算

□ 6.4.1 浮点加减运算

□ 6.4.2 浮点乘除法运算（自学）

□ 6.4.3 浮点运算所需的硬件配置（自学）



6.4.1 浮点加减运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

浮点加减运算步骤：

- (1) 对阶 对齐小数点，**小阶向大阶**看齐。
- (2) 尾数求和 **双符号位**的补码形式进行加减运算。
- (3) 规格化 将非规格化的结果值进行**规格化处理**
- (4) 结果舍入 尽量保持计算后数据的精确度
- (5) 溢出判断 以**阶码**的溢出为标准



1. 对阶

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y & \text{已对齐} \\ > 0 & j_x > j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1, j_x - 1 & \text{丢高位影响大} \\ y \text{ 向 } x \text{ 看齐} \checkmark & S_y \rightarrow 1, j_y + 1 & \text{丢低位影响小} \end{cases} \\ < 0 & j_x < j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} \checkmark & S_x \rightarrow 1, j_x + 1 & \text{丢低位影响小} \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1, j_y - 1 & \text{丢高位影响大} \end{cases} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐，尾数向右移位。

28. 浮点数的加减运算中，对阶时需要 [填小 1] 阶向 [填大 2] 阶看齐，即小阶的尾数向 [填右 3] 移位，每移一位，阶码 [填加 4] 1，直到两数的阶码相等为止。（填写汉字）



2. 尾数求和

对阶后的尾数按定点数加减运算规则求和（差）

以双符号位的补码形式进行加减法操作



3. 规格化

(1) 规格化数的定义

$$r = 2 \quad \frac{1}{2} \leq |S| < 1$$



(2) 规格化数的判断

$S > 0$

规格化形式

真值

$0.1 \times \times \dots \times$

原码

$0.1 \times \times \dots \times$

补码

$0.1 \times \times \dots \times$

$00.1 \times \times \dots \times$

反码

$0.1 \times \times \dots \times$

$S < 0$

规格化形式

真值

$-0.1 \times \times \dots \times$

原码

$1.1 \times \times \dots \times$

补码

$1.0 \times \times \dots \times$

$11.0 \times \times \dots \times$

反码

$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同

29. 在浮点机中，判断原码规格化形式的原则是()。

B

- ☐ A 尾数的符号位与第一数值位不同
- ☒ B 尾数的第一数值位为1
- ☐ C 尾数的符号位与第一数值位相同
- ☐ D 阶符与数符不同

提交



30. 在浮点机中，判断补码规格化形式的原则是()。

A

- ☒ A 尾数的符号位与第一数值位不同
- ☐ B 尾数的第一数值位为1
- ☐ C 尾数的符号位与第一数值位相同
- ☐ D 阶符与数符不同

提交





(3) 特例

$$S = -\frac{1}{2} = -0.100 \cdots 0$$

$$[S]_{\text{原}} = 1.100 \cdots 0$$

$$[S]_{\text{补}} = \boxed{1.1}00 \cdots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$S = -1$$

$$[S]_{\text{补}} = \boxed{1.0}00 \cdots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数

31. 下列形式的尾数S，符合规格化浮点数的是()。

D

- ☐ A $[S]_{\text{原}} = 1.000\ 1011$
- ☐ B $[S]_{\text{补}} = 1.100\ 1011$
- ☐ C $[S]_{\text{补}} = 0.000\ 1011$
- ☒ D $[S]_{\text{补}} = 1.000\ 0000$

提交



3. 规格化

(4) 左规

尾数出现 $00.0 \times \times \cdots \times$ 或 $11.1 \times \times \cdots \times$ 时

需左规

尾数左移一位，阶码减 1，直到数符和第一数位不同为止

(5) 右规

当尾数溢出 (>1) 时，需右规

即尾数出现 $01. \times \times \cdots \times$ 或 $10. \times \times \cdots \times$ 时

尾数右移一位，阶码加 1

32. 若浮点数的尾数使用双符号位补码，当尾数出现 $00.0 \times \times \dots \times$ 或 $11.1 \times \times \dots \times$ 形式时，需要进行 **左规**，当尾数出现 $01. \times \times \dots \times$ 或 $10. \times \times \dots \times$ 时，需要进行 **右规**。



4. 舍入

在**对阶**和**规格化**的过程中，尾数需要右移，可能出现尾数末位丢失，引起误差，需考虑舍入。

(1) 0 舍 1 入法

(2) 恒置“1”法

舍入可能导致尾数再次溢出，此时需再做一次右规。

33. 如果采用0舍1入法进行舍入处理，则0.01010110011舍去最后一位后，结果为()。

B

- ☐ A 0.0101011001
- ☒ B 0.0101011010
- ☐ C 0.0101011011
- ☐ D 0.0101011100

提交



34. 采用末位恒置1法进行舍入处理，则0.01010110011舍去最后一位后，结果为()。

A

- ☒ A 0.0101011001
- ☐ B 0.0101011010
- ☐ C 0.0101011011
- ☐ D 0.0101011100

提交



5. 溢出判断

在浮点运算中，根据阶码判断是否溢出

□ 上溢 阶码 $>$ 最大阶码

□ 下溢 阶码 $<$ 最小阶码 按 机器零 处理

设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码的数值部分取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为：

最大阶码：00, 111 1111

最大尾数：00. 1...1

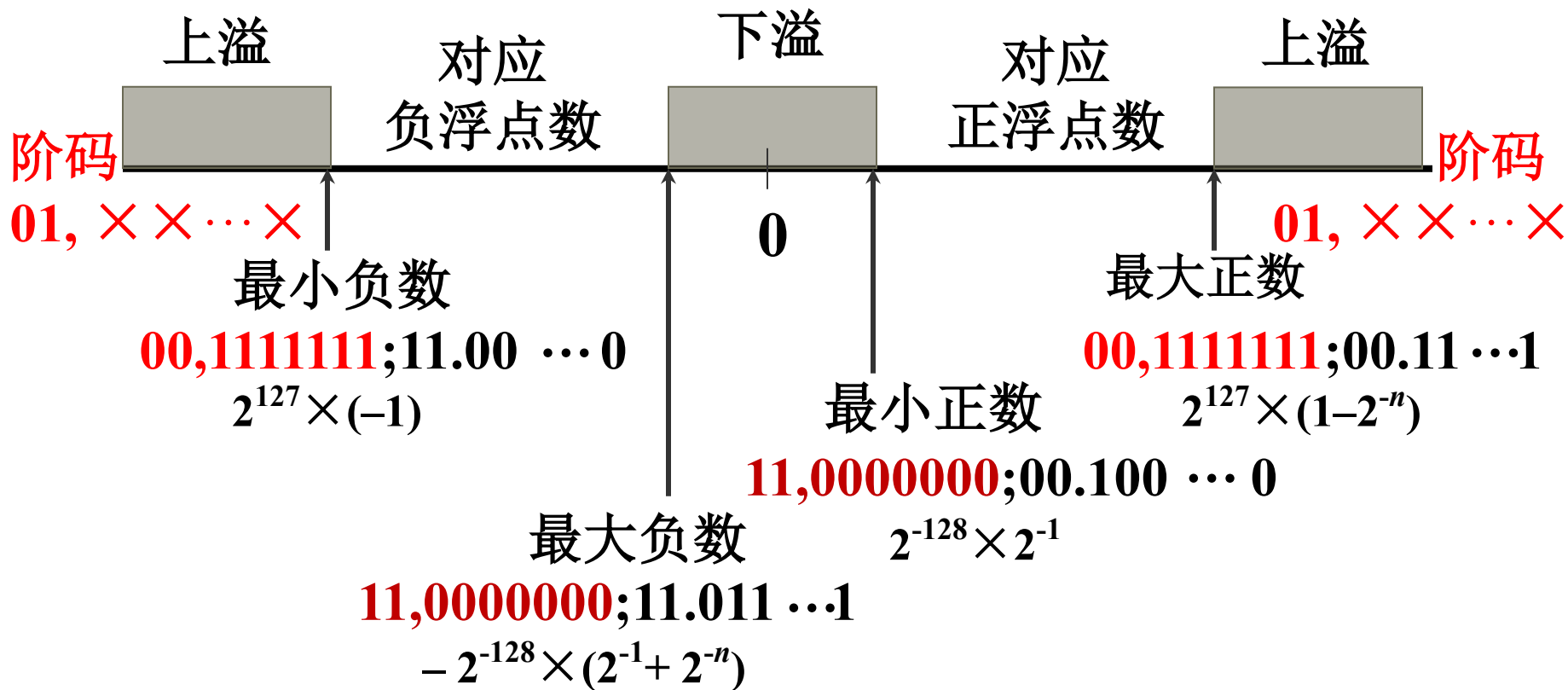
最小阶码：11, 000 0000

最小尾数：11. 000 0000



5. 溢出判断

阶码 10, $\times \times \dots \times$ 下溢，按机器零处理





5. 溢出判断

□ 尾数溢出

- 在规格化处理时，通过完成右规完成；

浮点溢出，均指阶码上溢。

□ 阶码溢出

- 上溢(结果绝对值太大)——置上溢标志，结束；
- 下溢(结果绝对值太小)——置机器零。

阶码 01, $\times \times \cdots \times$ 上溢

阶码 10, $\times \times \cdots \times$ 下溢，按机器零处理

35. 浮点数运算中，根据阶码判断是否溢出。若结果的阶码大于最大阶码，则产生 **上溢出** 若结果的阶码小于最小阶码，则产生下溢出，按 **[填空2]** 处理。

机器零

36. 【2015 浮点数运算】下列有关浮点数加减运算的叙述中，正确的是（ ）。

B

- ☐ A 对阶操作有可能会引起阶码上溢或下溢
- ☒ B 右规和尾数舍入都有可能会引起阶码上溢
- ☐ C 左规时不可能引起阶码下溢
- ☐ D 右规时不可能引起阶码上溢

提交





例6.29 假设浮点数用补码表示，阶符、数符各取2位，阶码取2位，尾数取4位。舍入采用末位恒置1法。已知 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$, 求 $x + y$ 。

解: $[x]_{\text{补}}$: 阶码 **00**, 10; 尾数 **00**. 1101

$[y]_{\text{补}}$: 阶码 **00**, 01; 尾数 **00**. 1011

① 对阶 $[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = [j_x]_{\text{补}} + [-j_y]_{\text{补}} =$

$$\begin{array}{r} 00, 10 \\ + 11, 11 \\ \hline \end{array}$$

阶差为 +1

舍掉 **1**00, 01

$\therefore S_y$ 右规1位: 00.0101 **(1)**, j_y+1

称为**保留附加位**，需专门寄存器。EEE754一般保留3个附加位。

对阶后 y : 阶码 **00**, 10; 尾数 **00**. 0101 **(1)**



例6.29 假设浮点数用补码表示，阶符、数符各取2位，阶码取2位，尾数取4位。舍入采用末位恒置1法。已知 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$ ，求 $x + y$ 。

解： ② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}} = 00.1101 \\ + [S_y]_{\text{补}} = 00.0101 \quad (1) \text{ 对阶后的 } [S_y]_{\text{补}} \\ \hline 01.0010 \quad (1) \end{array}$$

结果符号位出现01，尾数溢出，需右规



例6.29 假设浮点数用补码表示，阶符、数符各取2位，阶码取2位，尾数取4位。舍入采用末位恒置1法。已知 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$, 求 $x + y$ 。

解：

③ 右规

$[x + y]_{\text{补}}$: 阶码 00, 10; 尾数 01. 0010 (1)

右规后: $[x + y]_{\text{补}}$: 阶码 00, 11; 尾数 00. 1001 (01)



例6.29 假设浮点数用补码表示，阶符、数符各取2位，阶码取 2 位，尾数取 4 位。舍入采用末位恒置1法。已知 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$, 求 $x + y$ 。

解：

④ 舍入处理

采用 末位恒置1，可得：

$[x + y]_{\text{补}}$ ：阶码 **00**, 11; 尾数 **00**. 1001



例6.29 假设浮点数用补码表示，阶符、数符各取2位，阶码取2位，尾数取4位。舍入采用末位恒置1法。已知 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$ ，求 $x + y$ 。

解：

⑤ 溢出判断

$x + y$ 阶码 00,11，未溢出

$$x + y = 0.1001 \times 2^{11}$$



6.5 算术逻辑单元（实验课中进行）

□ 6.5.1 ALU电路（自学）

□ 6.5.2 快速进位链

0. 全加器

1. 并行加法器

2. 串行进位链

3. 并行进位链

本章重点

1. 无符号数和有符号数的表示
2. 定点数和浮点数的表示
3. IEEE754标准
4. 定点数移位、加减运算
5. 浮点数加减运算

